Trí tuệ nhân tạo (Artificial Intelligence)

Các chiến lược tìm kiếm có đối thủ (trò chơi game)

Giới thiệu

- Nghiên cứu máy tính chơi cờ xuất hiện từ rất sớm
- Không lâu sau khi máy tính lập trình ra đời, năm 1950, Claude Shannon đã viết chương trình chơi cờ đầu tiên
- Việc máy tính chơi cờ là một bằng chứng về khả năng máy tính có thể làm những công việc đòi hỏi trí thông minh của con người

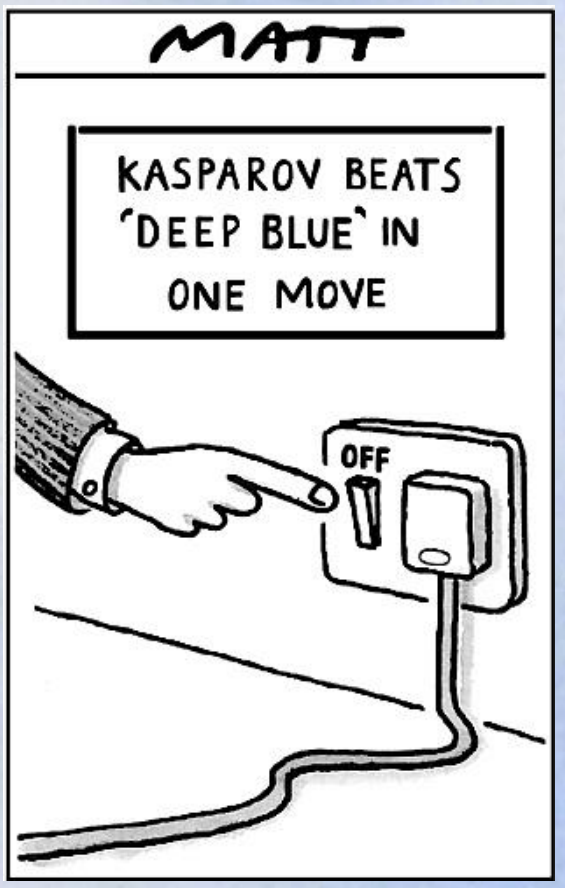
Ví dụ



Ví dụ



World Champion chess player Garry Kasparov is defeated by IBM's Deep Blue chess-playing computer in a six-game match in May, 1997



© Telegraph Group Unlimited 1997

Ví dụ



AlphaGo beats Lee Sedol, the world's top Go player 4-1 in March, 2016

- Using Deep Learning
- The human brain has 1,000 trillion synapses, while current neural networks only have about 1 billion synapses.

Types of games

	Deterministic	Chance
Perfect information	**Chess**, checkers, go, othello	Backgammon, monopoly
Imperfect information	Sea battle	Bridge, **poker**, scrabble, nuclear war

Why Games?

- “Unpredictable” opponent: solution is a strategy:
  - Must respond to every possible opponent reply
- Time limits: must rely on approximation
  - Tradeoff between speed and accuracy
- Games have been a key driver of new techniques in CS and AI.

Tìm kiếm có đối thủ

- Sẽ nghiên cứu các trò chơi có 2 người tham gia
- Xem xét các vấn đề:
  - Chơi cờ có thể xem như vấn đề tìm kiếm trong không gian trạng thái
  - Chiến lược tìm kiếm nước đi Minimax
  - Phương pháp cắt cụt α-β: α-β pruning

Bài toán

- Giả sử một người chơi cầm quân Trắng, một người chơi cầm quân Đen – nghiên cứu để máy tính cầm quân Trắng
- Nghiên cứu các trò chơi:
  - Hai người thay phiên nhau đưa ra các nước đi tuân theo luật đi nào đó.
  - Các luật này là như nhau cho cả hai người
  - Biết được thông tin đầy đủ và các tình thế trong trò chơi (không giống như chơi bài)
  - Ví dụ: Cờ vua, cờ tướng
- Bài toán: Tìm kiếm nước đi trong các nước đi có thể để thắng

Không gian trạng thái cho trò chơi

- Vấn đề chơi cờ có thể xem như vấn đề tìm kiếm trong không gian trạng thái:
  - Mỗi trạng thái là một tình thế
  - Trạng thái ban đầu là tình thế lúc bắt đầu cuộc chơi
  - Các toán tử là các nước đi hợp lệ
  - Các trạng thái kết thúc là các tình thế mà cuộc chơi dừng
  - Một hàm kết cuộc ứng với mỗi trạng thái kết thúc với một giá trị nào đó, ví dụ thắng là 1, thua là -1, hòa là 0

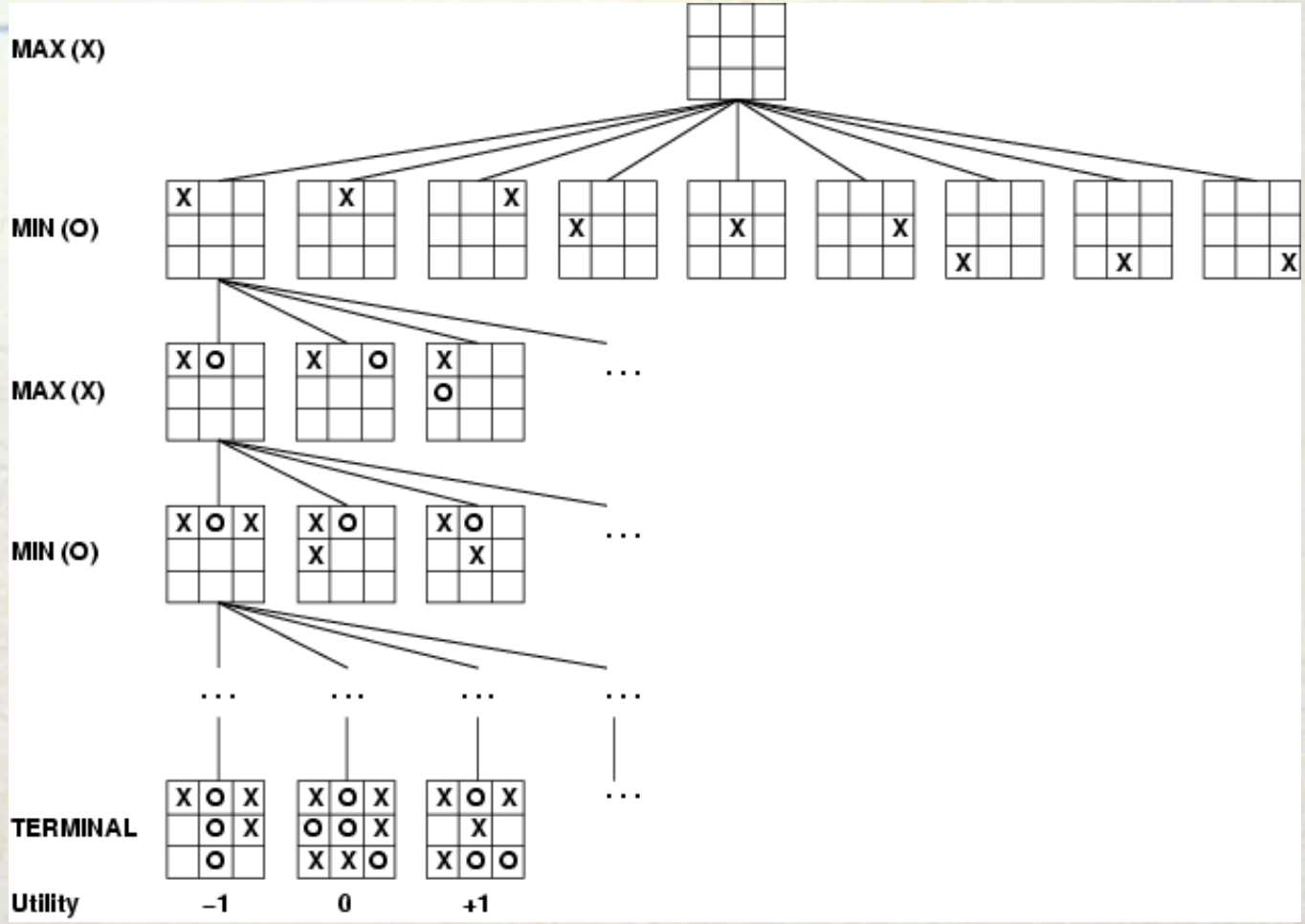
Không gian trạng thái cho trò chơi

- Vấn đề của Trắng là tìm một dãy nước đi sao cho xen kẽ với các nước đi của Đen tạo thành một đường đi từ trạng thái ban đầu tới giá trị kết cuộc là cao nhất
- Không biết trước nước đi của đối thủ:
  - liệt kê hết các nước đi của đối thủ.

Cây trò chơi

- Để thuận lợi, ta biểu diễn không gian trạng thái dưới dạng cây trò chơi
- Cây trò chơi được xây dựng như sau:
  - Gốc của cây ứng với trạng thái ban đầu
  - Gọi đỉnh ứng với trạng thái mà Trắng (Đen) sẽ đưa ra nước đi là đỉnh Trắng (Đen)

Game Tree
2-player, deterministic



Chiến lược Minimax

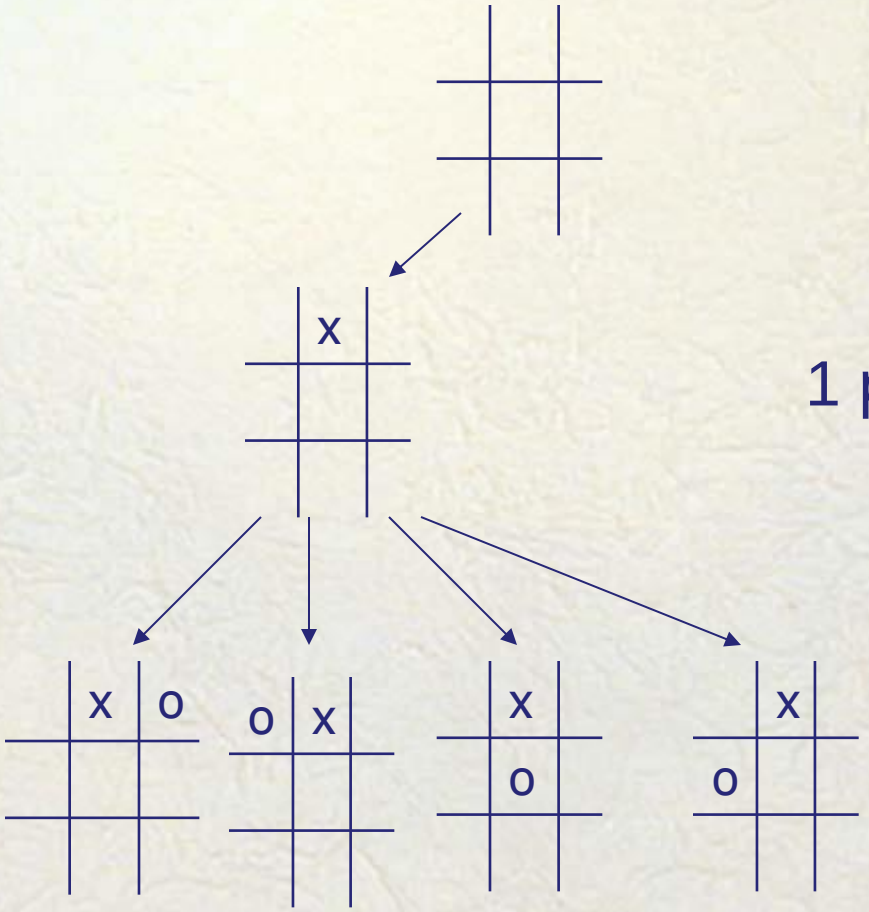
- Perfect play for deterministic environments with perfect information
- **Basic idea:** choose move with highest minimax value
  = best achievable payoff against best play
- **Algorithm:**
  1. Generate game tree completely
  2. Determine utility of each terminal state
  3. Propagate the utility values upward in the three by applying MIN and MAX operators on the nodes in the current level
  4. At the root node use minimax decision to select the move with the max (of the min) utility value

- Steps 2 and 3 in the algorithm assume that the opponent will play perfectly.

Chiến lược Minimax

- Chọn nước đi với giá trị minimax lớn nhất:
  - Trắng = Max, Đen = Min.
- Đi ngược từ các trạng thái kết thúc
- Gán giá trị cho các trạng thái kết thúc là giá trị của hàm kết cuộc
- Đi ngược từ dưới lên:
  - nếu là đỉnh trắng thì gán giá trị là max của giá trị những nút con của nó;
  - nếu là đỉnh đen thì gán giá trị là min của giá trị những nút con của nó
- Trắng: chọn nước đi là nút con có giá trị lớn nhất

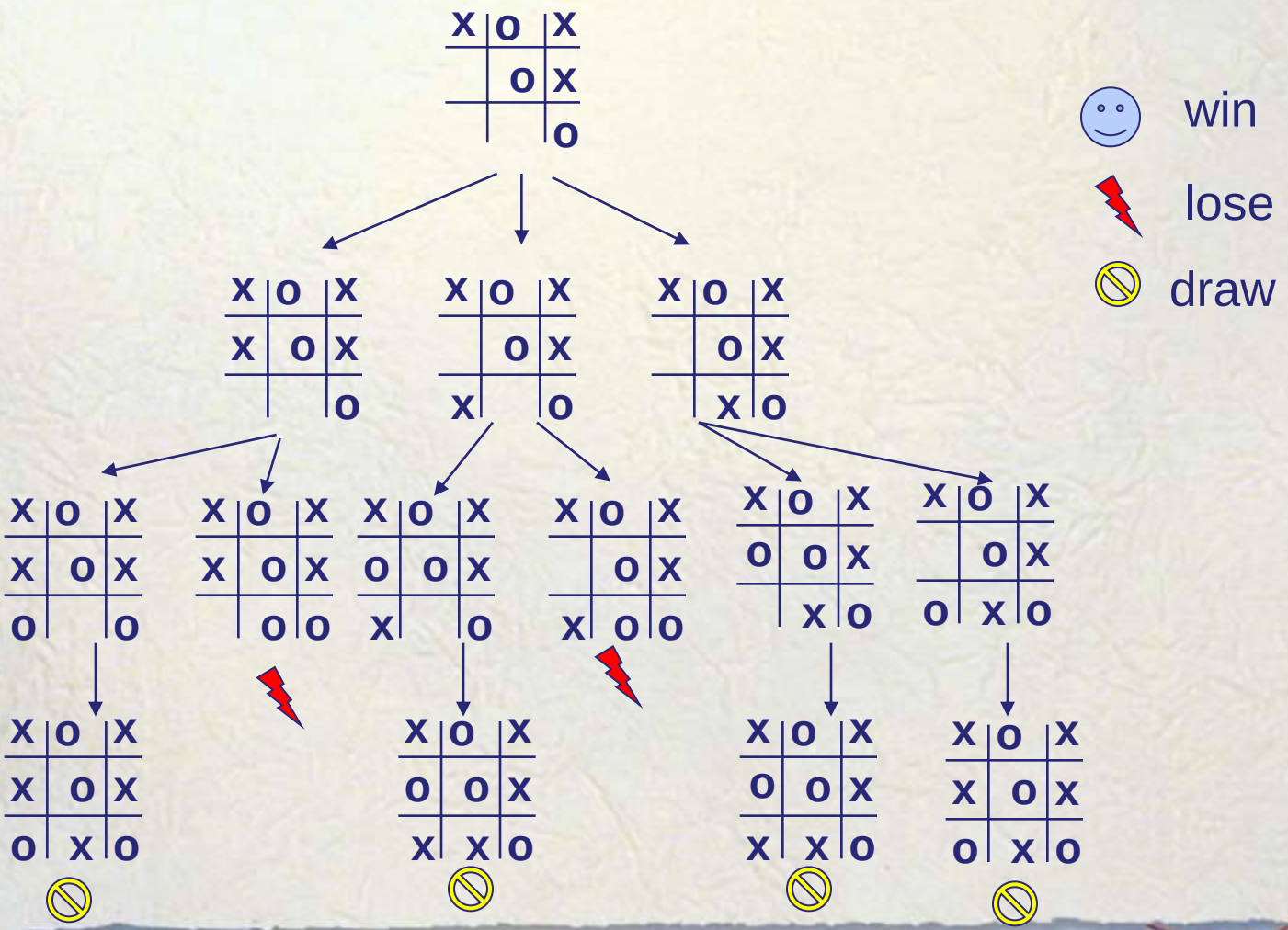
Generate Game Tree



1 ply

1 move

A subtree

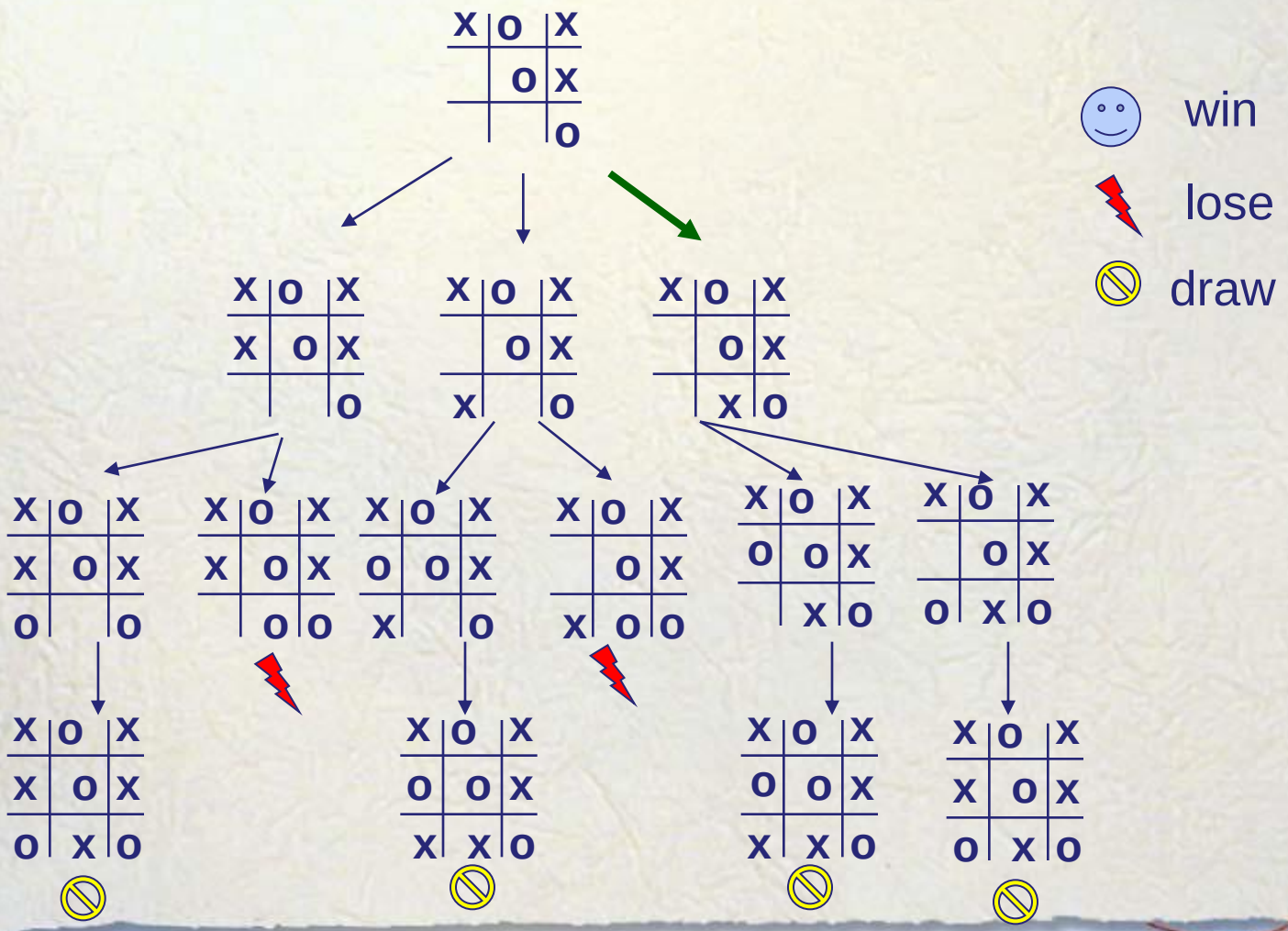


win

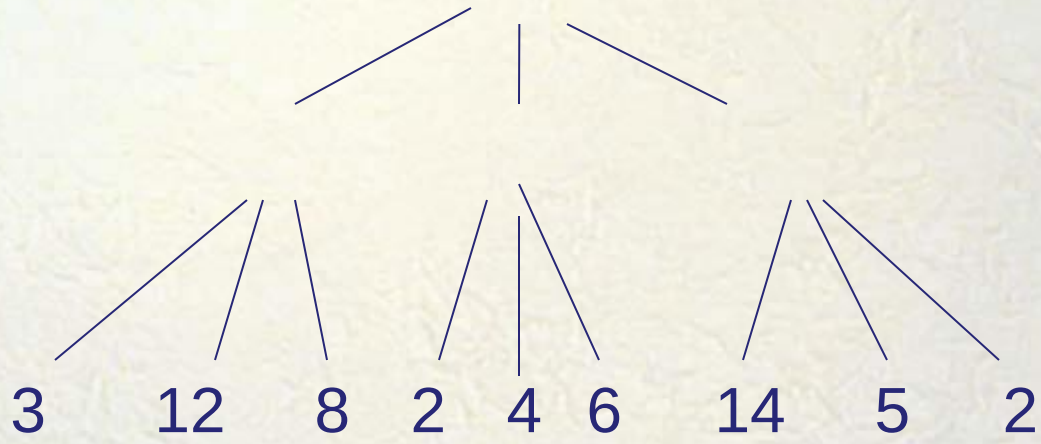
lose

draw

What is a good move?

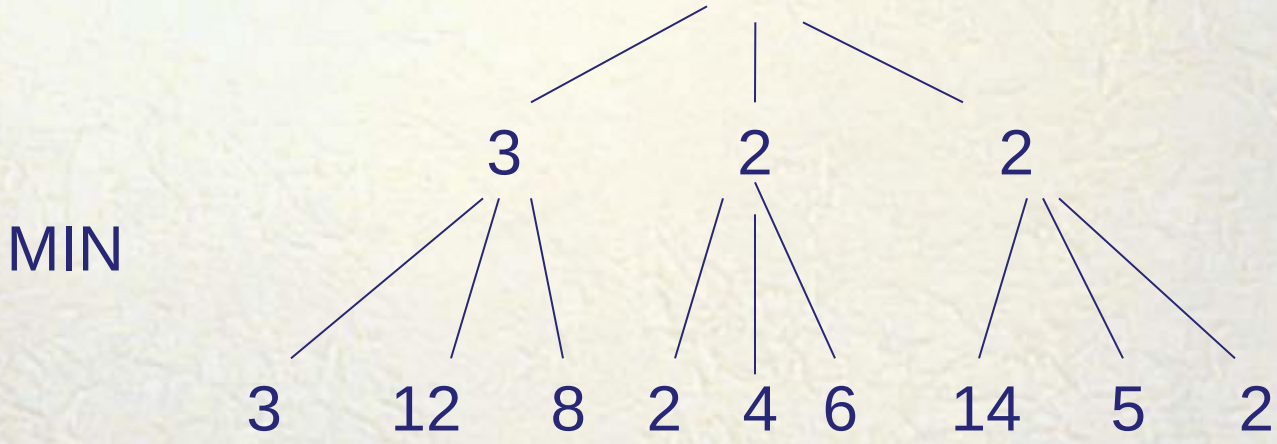


Minimax



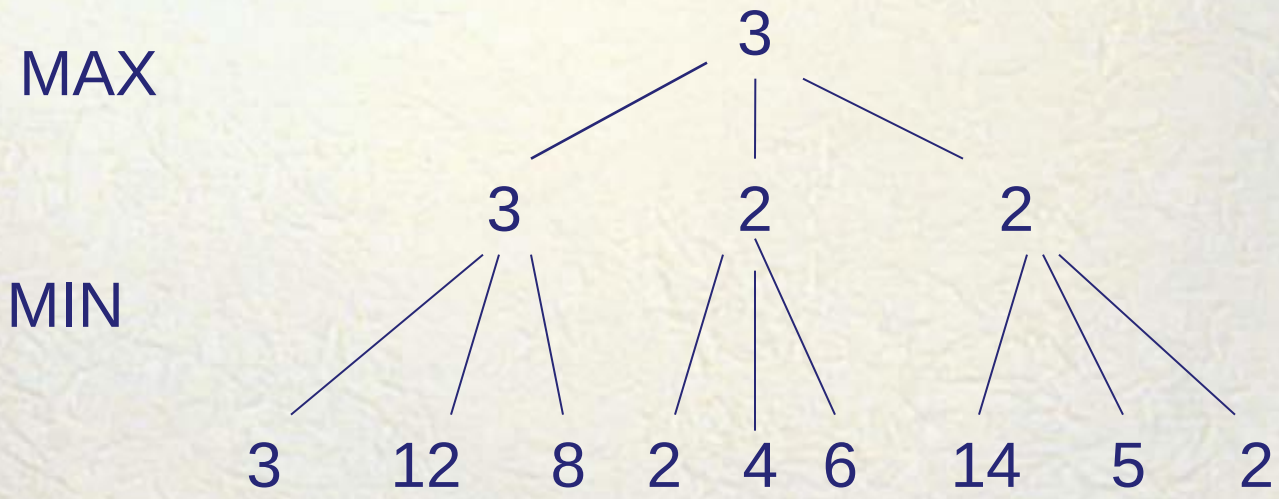
- Minimize opponent's chance
  - Maximize your chance

Minimax



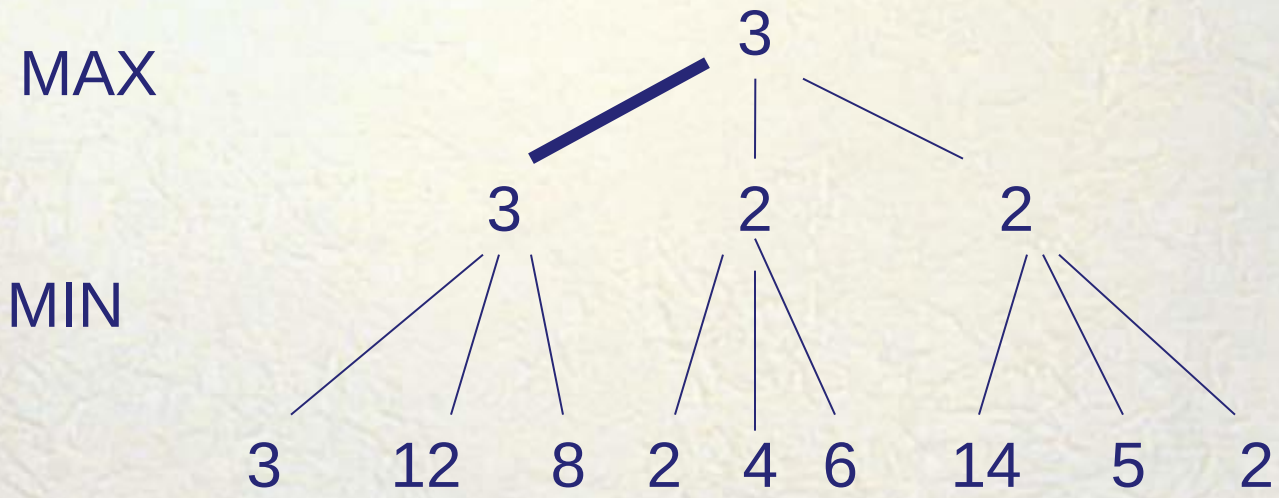
- Minimize opponent's chance
  - Maximize your chance

Minimax



- Minimize opponent's chance
  - Maximize your chance

Minimax



- Minimize opponent's chance
  - Maximize your chance

Chiến lược Minimax (…)

```
Function minimax (current_node)
{
     if is_leaf (current_node)
          then return static_evaluation (current_node);
     if is_min_node (current_node)
          then return min (minimax (children_of
          (current_node)));
     if is_max_node (current_node)
          then return max (minimax (children_of
          (current_node)));
     // this point will never be reached since
     // every node must be a leaf node, a min node or a
     // max node.
}
```

Chiến lược Minimax (…)

- Là thuật toán tìm kiếm theo độ sâu
- Cho phép ta chọn được nước đi tối ưu
- Tuy nhiên độ phức tạp quá lớn
- Có thể hạn chế độ sâu của cây trò chơi và sử dụng hàm đánh giá để giảm bớt cây tìm kiếm

Chiến lược Minimax (…)

```
Function bounded_minimax (current_node, max_depth)
{
     if is_leaf (current_node)
          then return static_evaluation (current_node);
     if depth_of (current_node) == max_depth
          then return static_evaluation (current_node);
     if is_min_node (current_node)
          then return min (minimax (children_of
          (current_node)));
     if is_max_node (current_node)
          then return max (minimax (children_of
          (current_node)));
     // this point will never be reached since
     // every node must be a leaf node, a min node or a
     // max node.
}
```

Đánh giá Minimax

- Complete?
- Optimal?
- Time complexity?
- Space complexity?
-

Đánh giá Minimax

- Complete? Yes (if tree is finite)
- Optimal? Yes (against an optimal opponent)
- Time complexity? $O(b^m)$
- Space complexity? O(bm) (depth-first exploration)
- Chơi cờ vua: b ~ 35, m ~100: tìm nghiệm tối ưu là không khả thi.
  - $b^m = 10^6$ b= 35 -> m = 4
  - 4-ply = người mới chơi
  - 8-ply = máy tính thông thường, người chơi giỏi
  - 12-ply= Deep Blue, Kasparov

Hàm đánh giá

- Thường là giá trị đánh giá độ lợi thế của trạng thái u – eval(u)
- Trạng thái u càng thuận lợi cho Trắng thì eval(u) là số dương càng lớn: MAX
- Trạng thái u càng thuận lợi cho Đen thì eval(u) là số âm càng nhỏ: MIN
- eval(u) ≈ 0 đối với trạng thái u không lợi thế cho ai cả
- eval(u) = rất lớn – Trắng thắng
- eval(u) = rất nhỏ - Đen thắng

Hàm đánh giá (…)

- Chất lượng của chương trình chơi cờ rất phụ thuộc vào hàm đánh giá
- Tuy nhiên độ tốt của một hàm đánh giá thường mâu thuẫn với thời gian để tính nó

Hàm đánh giá (…)

Ví dụ: Cờ vua, cờ tướng

- Mỗi loại quân gán cho một giá trị phù hợp với sức mạnh, ví dụ Tốt (1), Mã (5) …rồi lấy tổng
- Có thể xét đến mối tương quan vị trí giữa các quân cờ

Pruning - Motivation



- Q1. Why would “Queen to G5” be a bad move for Black?
- Q2. How many White “replies” did you need to consider in answering?

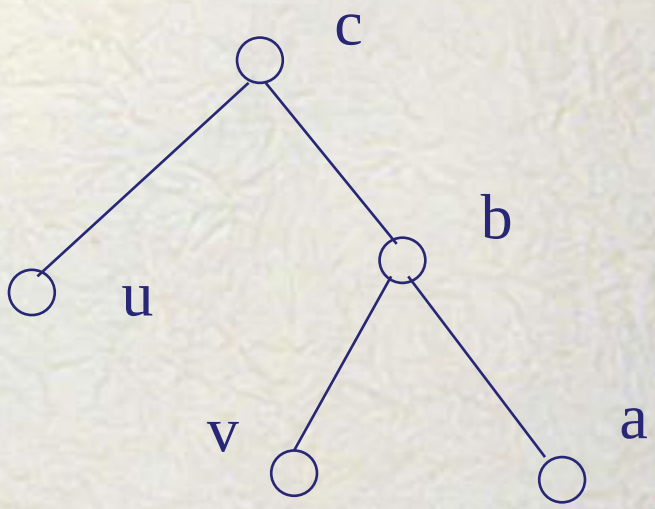
Once we have seen one reply scary enough to convince us the move is really bad, we can abandon this move and continue searching elsewhere.

Phương pháp cắt cụt alpha-beta

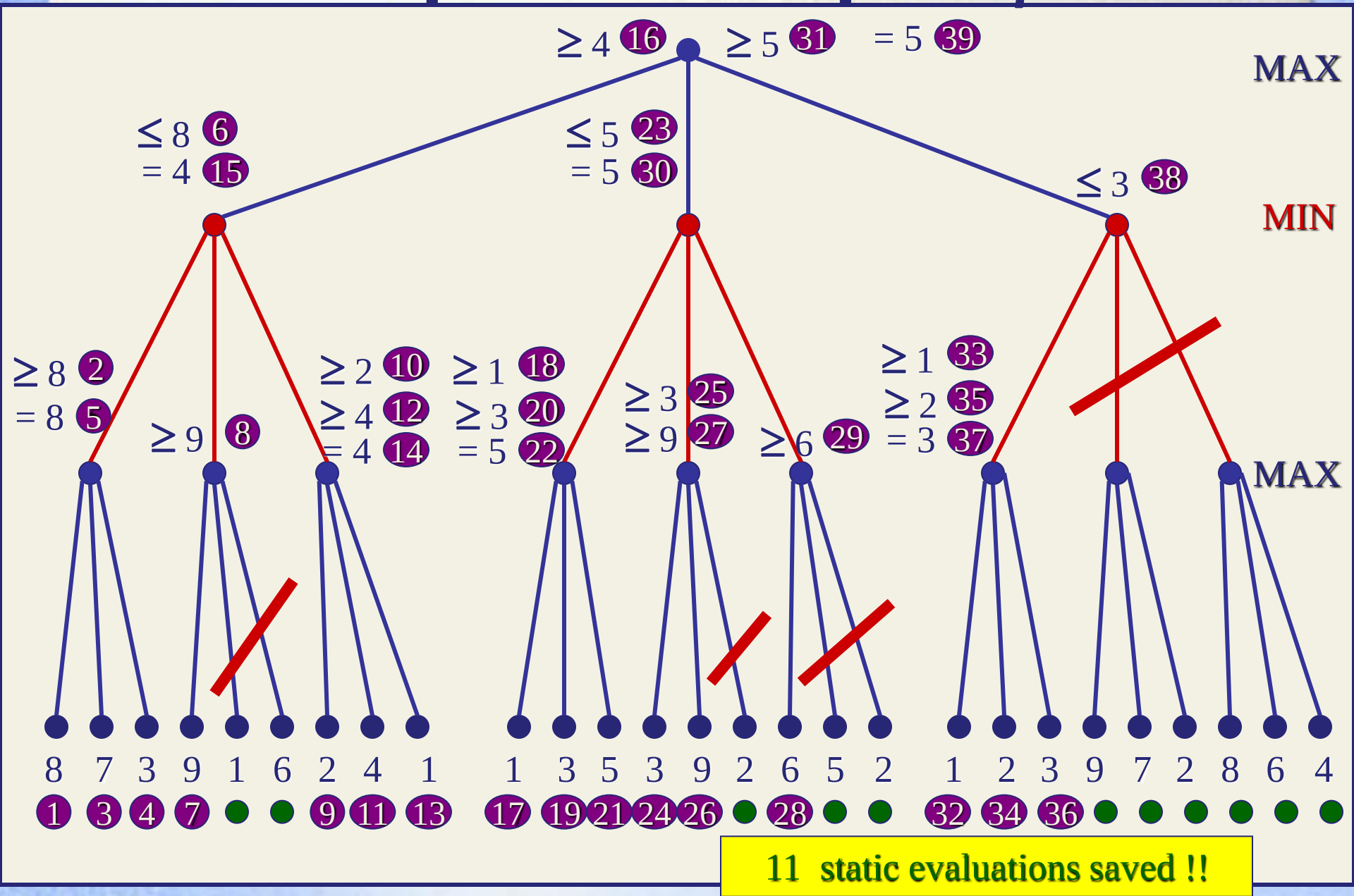
- Cho phép cắt bỏ các nhánh không cần thiết cho sự đánh giá đỉnh u

Nếu a là đỉnh trắng, nếu u, v đã được gán và giá trị của u > giá trị của v thì không cần gán giá trị của a nữa

Nếu a là đỉnh đen, nếu u, v đã được gán và giá trị của u < giá trị của v thì không cần gán giá trị của a nữa



Ví dụ với cắt cụt α–β

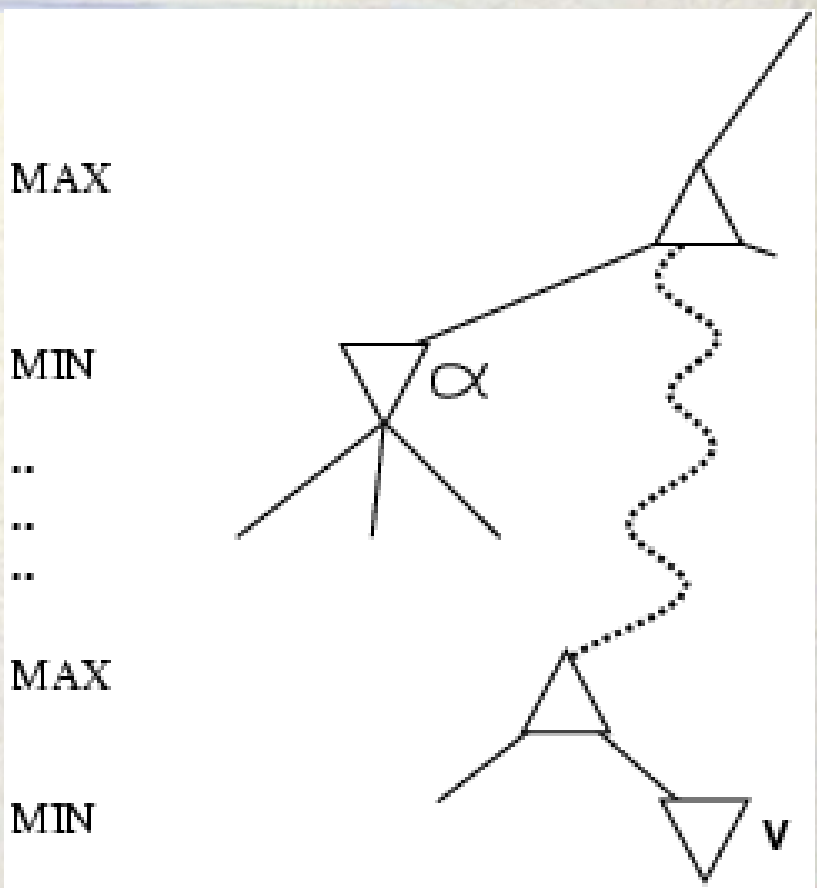


Phương pháp cắt cụt alpha-beta

- Phương pháp cắt cụt alpha-beta không ảnh hưởng đến kết qủa cuối cùng, chỉ ảnh hưởng đến thời gian tìm kiếm.
- Thứ tự sắp xếp các bước đi trong cây tìm kiếm ảnh có ảnh hưởng lớn đến "chất lượng" của phương pháp cắt cụt alpha-beta.
- Với một "sắp xếp hoàn hảo", time complexity = $O(b^{m/2})$ từ $O(b^m)$.

Tại sao lại gọi là α-β?

- α là giá trị tốt nhất (i.e., giá trị cao nhất) tính đến thời điểm hiện tại cho *max*
- Nếu *v kém hơn* α, *max* sẽ loại bỏ v
  → cắt nhánh
- Định nghĩa β tương tự cho *min*
-



Phương pháp cắt cụt anpha-beta

Function MaxVal(u,α,β);

Begin

If u là lá của cây hạn chế hoặc là đỉnh kết thúc **then**

MaxVal ← eval(u)

else for (mỗi đỉnh v là con của u) **do**

{ α ← max[α,MinVal(v, α,β)];

if α ≥ β then exit;}

//Cắt bỏ các cây con từ các đỉnh v còn lại

MaxVal ← α

End;

Phương pháp cắt cụt anpha-beta

Function MinVal(u,α,β);

Begin

If u là lá của cây hạn chế hoặc là đỉnh kết thúc **then**

Minval ← eval(u)

else for (mỗi đỉnh v là con của u) **do**

{ β← min[β, MaxVal(v, α,β)];

if α ≥ β then exit;}

//Cắt bỏ các cây con từ các đỉnh v còn lại

MinVal ← β

End;

Phương pháp cắt cụt anpha-beta

Procedure Alpha_beta(u,v);
Begin
    α ← -∞
    β ← ∞
    For mỗi đỉnh w là con của u **do**
        if α ≤ MinVal(w, α,β) **then**
        { α ← MinVal(w, α,β);
        v ← w;}
End;

Advanced techniques

- **Transposition table** to store previously expanded states
- **Forward pruning** to avoid considering all possible moves
- **Lookup tables** for opening moves and endgames

Hệ thống chơi cờ

- Baseline system: 200 million node evaluations per move (3 min), minimax with a decent evaluation function and quiescence search
  - 5-ply ≈ human novice
- Add alpha-beta pruning
  - 10-ply ≈ typical PC, experienced player
- Deep Blue: 30 billion evaluations per move, singular extensions, evaluation function with 8000 features, large databases of opening and endgame moves
  - 14-ply ≈ Garry Kasparov
- More recent state of the art (Hydra, ca. 2006): 36 billion evaluations per second, advanced pruning techniques
  - 18-ply ≈ better than any human alive?

Go



Checkers



Robocup Soccer



Deep Green chơi bi-a



Deterministic games in practice

- Checkers: Chinook ended 40-year-reign of human world champion Marion Tinsley in 1994. Used a pre-computed endgame database defining perfect play for all positions involving 8 or fewer pieces on the board, a total of 444 billion positions.
- Chess: Deep Blue defeated human world champion Garry Kasparov in a six-game match in 1997. Deep Blue searches 200 million positions per second, uses very sophisticated evaluation, and undisclosed methods for extending some lines of search up to 40 ply.
- Othello: human champions refuse to compete against computers, who are too good.
- Go: AlphaGo beats Lee Sedol, the world's top Go player 4-1.
-

Tóm tắt

- Các bài toán trò chơi có đối thủ rất thú vị.
- Chơi cờ có thể xem như vấn đề tìm kiếm trong không gian trạng thái
- Chiến lược tìm kiếm nước đi Minimax
- Phương pháp cắt cụt α-β

References

- Artificial Intelligence: A modern approach. Chapter 6.
- Artificial Intelligence Illuminated. Chapter 6.

Kiểm tra giữa kỳ

- Thời gian: Thứ ba, 13:00, 17/10/2017
- Nội dung: Bài 1 – 5
- Hình thức:
  - Thi viết
  - Không mở sách
  - Được chuẩn bị tài liệu là 1 mặt giấy A4 tự viết tay bằng bút bi xanh, k được mang bản copy

Làm gì khi nghỉ học ở lớp?

- Thưa cô! Em là .... Ngày mai em có lịch học bù Trí Tuệ Nhân Tạo của cô nhưng mai e trùng lịch học môn Xác Suất Thống Kê nên em xin nghỉ buổi học bù ngày mai ạ. Mong cô thông cảm. Em cảm ơn cô!
- Em chào cô, Em là ..., tham gia lớp học phần Trí tuệ nhân tạo INT3401 2 do cô giảng dạy. Em viết mail này xin cô được nghỉ buổi học bù sáng nay do bị ốm ạ. Em cảm ơn cô ạ. Trân trọng,
- ...



Em chào cô ạ! Em học lớp trí tuệ nhân tạo của cô. Lớp mình có học bù sáng nay nhưng em và một số bạn trùng lịch học chính ở trường. Em muốn thắc mắc tại sao lại điểm danh và có thể cộng điểm bạn đi hôm nay ạ? cho những

Hi ..., Việc em và các bạn trùng lịch mình đã dặn mọi người cần viết email cho mình khi có lý do chính đáng. Không thể lẫn lộn người bị trùng lịch học với người tự ý nghỉ học được. Ngoài ra, về việc cộng điểm em quên là có điểm chuyên cần. Mọi người nghỉ học tất nhiên là điểm này kém hơn người đi học đầy đủ. Thân,

Thưa cô, em biết là có điểm chuyên cần và tất nhiên đi học đầy đủ sẽ được tính điểm. Nhưng buổi học hôm trước là học bù do cô có việc bận, mà xếp lịch học bù lại trùng với lịch chính của bọn em. Bọn em phải ưu tiên lịch học chính. Việc điểm danh hôm đó để tính vào điểm chuyên cần em thấy thật vô lý. Em mong cô có thể hủy bản điểm danh hôm đó và chỉ điểm danh vào những hôm học chính thôi ạ! Em cảm ơn cô!

